

CONTEXTO DEL PROYECTO

Estás trabajando en **Che.Comex (ComexIA)** — plataforma SaaS B2B de comercio exterior para PyMEs argentinas. Stack: React + Vite (port 5174), Express + TypeScript backend (port 3001), SQLite (comexia_v2.db), Drizzle ORM.

Design system ya definido y NO negociable:

- Fondo: #020a12, Cyan: #00d4f0, Green: #00e878, Amber: #f5a800, Red: #ff3e5a, Gold: #f0c040
- Fonts: **Barlow Condensed** (display/títulos) + **DM Mono** (datos/labels) + **Outfit** (body)
- Estética: Bloomberg/fintech dark — densa, datos visibles, sin espacios muertos

APIs ya construidas y funcionales en el proyecto:

POST /api/onboarding/requirements → docs requeridos por país+rol+subRol (cache SQLite + Groq fallback)

GET /api/logistics/calculate → costo CIF/FOB por ruta + aranceles por NCM

POST /api/ai/chat → Groq llama-3.3-70b con contexto de comercio AR

GET /api/onboarding/plans → planes por role

PATCH /api/onboarding/verify-doc → actualiza score de verificación

Tablas SQLite existentes:

onboarding_profiles → userId, role, country, docs_completed (JSON), verify_score, plan

onboarding_requirements_cache → cache_key (AR-trader-exporter), requirements (JSON)

Estado del usuario logueado disponible en contexto React:

```
interface UserProfile {  
  userId: string;  
  role: 'trader' | 'logistics' | 'institutional';  
  subRole: 'exporter' | 'importer' | 'both';  
  country: string; // "AR", "BR", etc.  
  company: string;  
  fiscalId: string;  
  industry: string;  
  plan: 'demo' | 'pro' | 'enterprise';  
  verifyScore: number; // 0-100  
  docsCompleted: string[]; // ["Registro AFIP (RIE)", "Firma Digital AFIP", ...]  
}
```

LO QUE HAY QUE CONSTRUIR — 4 SECCIONES EN ORDEN

SECCIÓN 1 — Marketplace Feed `/marketplace`

Archivo: `src/pages/MarketplacePage.tsx`

Layout 3 columnas:

Columna izquierda (240px) — Filtros Inteligentes:

- Header: "🔍 Filtros Inteligentes"
- Tipo: botones toggle Compra / Venta (selección única)
- Sector: lista con iconos — Agricultura 🌾, Tecnología 💻, Transporte 🚢, Manufactura 🏭, Servicios 📄, Ecológico 🌿
- País / Bloque: dropdown con banderas — Todos, 🇦🇷 Argentina, 🇧🇷 Brasil, 🇨🇳 China, 🇩🇪 Alemania, 🌐 MERCOSUR, 🇪🇺 UE, 🇦🇸 ASEAN
- Empresa: input de búsqueda
- Checkbox: "Solo verificadas ✓"
- Sección colapsable "AVANZADO": rango de precio USD, rango de cantidad, Incoterm específico, NCM/HS Code

Columna central (flex 1) — Feed:

Barra superior de publicar:

[Avatar J] [¿Qué oportunidad comercial querés compartir? _____]
[+ Agregar HS Code] [+ Documentos] [+ Contacto]

Cada **PublicationCard** contiene:

[Avatar empresa] [Nombre empresa] [badge VERIFICADA o PENDIENTE]
[Nombre contacto] [Cargo] [tiempo: "hace X"]
[badge VENDO/COMPRO] [N docs] [indicador compatibilidad]

[Título del producto + NCM en negrita grande]
[Certificaciones requeridas en texto]

[Cantidad] [Incoterm] [Precio USD] [País Origen bandera]

Certificaciones: [chip SENASA] [chip Orgánico] [chip ISO 9001]

[btn Contactar Vendedor] [btn Ver Costos →]

Indicador de compatibilidad documental (lógica crítica):

- Al renderizar cada card, comparar `userProfile.docsCompleted` con los requisitos de la ruta `origen→destino+NCM`
- Verde  "Podés operar": usuario tiene ≥80% de los docs requeridos
- Amarillo  "Te faltan N docs": usuario tiene 40-79%
- Rojo  "No podés operar esta ruta": usuario tiene <40%
- Esta comparación usa el cache local de `onboarding_requirements_cache` — NO llama a la IA en cada render, solo lee la DB

Columna derecha (280px) — World Trade Pulse:

- Sección "World Trade Pulse" con link "Ver todo"
- Lista de noticias: [título] [tag tipo: Regulación/Tratado/Alerta] [tiempo]
- Sección "Eventos de Comercio": [nombre evento] [fecha · lugar]
- Sección "Grupos Sugeridos": [avatar] [nombre grupo] [btn +]

Mock data para el feed (5 publicaciones de ejemplo):

```
const MOCK_PUBLICATIONS = [
  {
    id: '1',
    company: 'Empresa Comercial 14',
    contact: 'Miguel Martínez',
    role: 'Export Manager',
    verified: true,
    type: 'sell', // 'sell' | 'buy'
    product: 'Porotos de Soja Orgánica Premium',
    hsCode: '120190',
    qty: 1000,
    unit: 'toneladas',
    incoterm: 'FOB',
    price: 450,
    currency: 'USD',
    origin: 'BR',
    certifications: ['SENASA', 'Orgánico Certificado', 'ISO 9001'],
    docsCount: 6,
    timeAgo: 'hace menos de 1 hora',
  },
  // ... 4 más variados: trigo AR→CN, vino Mendoza→DE, aceite girasol→UE,
  // manufactura→US
];
```

SECCIÓN 2 — Modal Análisis de Costos y Documentación

Archivo: `src/components/marketplace/CostAnalysisModal.tsx`

Props:

```
interface Props {
  isOpen: boolean;
  onClose: () => void;
  publication: Publication; // la card que disparó el modal
  userProfile: UserProfile; // perfil del usuario logueado
}
```

Estructura del modal (pantalla completa o overlay grande ~860px):

Header:

\$ Análisis de Costos y Documentación [X cerrar]
 Porotos de Soja Orgánica Premium
 HS Code: 120190 · 1000 toneladas · BR → CN

Sección 1 — Cumplimiento Documental:

Badge: "57% Completo" (calculado dinámicamente)

Lógica:

```
// 1. Obtener requisitos de la ruta
const requirements = await fetch('/api/onboarding/requirements', {
  method: 'POST',
  body: JSON.stringify({
    country: publication.origin, // "BR"
    role: userProfile.role,      // "trader"
    subRole: userProfile.subRole // "exporter"
  })
});

// 2. Comparar con docs del usuario
requirements.forEach(req => {
  const status = userProfile.docsCompleted.includes(req.name)
    ? 'available' // ✅ Disponible
    : 'missing';  // ⚠️ Faltante
});
```

Cada fila del checklist:

[✅ verde / ⚠️ amarillo] [Nombre del documento] [Disponible / Faltante]

Loading state: spinner + "Analizando requisitos legales para BR → CN..."

Banner si hay docs faltantes:



Documentos Faltantes

Te faltan N documentos para completar el proceso de exportación.
Contactá al vendedor para coordinar.

Sección 2 — Desglose de Costos:

Llamar a GET

`/api/logistics/calculate?origin=BR&destination=CN&hsCode=120190&qty=1000&incoterm=FOB`

Mostrar tabla de costos:

Valor del Producto	\$62,500
 Transporte Marítimo	\$3,200
Seguro de Carga	\$625
Aranceles Aduanales (GSP 4%)	\$2,500
Manejo y Almacenaje	\$450
 Documentación	\$300

Costo Total Landed (DAP Miami) \$69,575 ← en cyan/destacado

Costo por tonelada: \$13,915

Botones footer:

[Cerrar] [Contactar Vendedor →] ← este abre el chat

Loading state completo: mientras carga la IA y los costos, mostrar skeleton loader con texto "Calculando costos y verificando documentación..."

SECCIÓN 3 — Chat Colaborativo `/chat/:dealId`

Archivo: `src/pages/ChatPage.tsx`

Layout:

Panel izquierdo (260px) — Lista de chats:

- Cada ítem: [avatar empresa] [nombre deal] [último mensaje truncado] [tiempo]
- Badge de mensajes no leídos
- Ítem activo destacado con borde cyan

Centro — Área de mensajes:

Header del chat activo:

← [nombre del deal / producto] [ pin] [ llamada] [ + agregar] [ grupo]

Área de mensajes:

- Mensajes propios: alineados derecha, fondo `rgba(0, 212, 240, .1)`
- Mensajes otros: alineados izquierda, fondo `var(--bg3)`
- Cada mensaje: [avatar] [nombre + cargo] [texto] [hora]
- Estado vacío: "No hay mensajes aún. ¡Iniciá la conversación!"

Input inferior:

[ doc] [ adjunto] [ resumen] [Escribí un mensaje...] [ enviar]

Panel derecho (280px) — 4 tabs:

Tab  Participantes:

- Lista: [avatar] [nombre] [cargo/rol] — Juan Pérez (Export Manager), Maria Chen (Buyer), Carlos Ruiz (Logistics)
- Btn "+ Invitar Usuario"

Tab  Documentos:

- Lista de docs subidos en el deal: nombre, quién subió, fecha, btn descargar

Tab ▶ Timeline:

- Estados del deal en vertical:
 -  Contacto iniciado
 -  Propuesta enviada
 -  Documentación (activo)
 -  Negociación
 -  Cierre

Tab  Resumen:

- Producto, NCM, cantidad, precio acordado, Incoterm, fecha estimada embarque

Mock de mensajes iniciales: 3-4 mensajes de ejemplo para mostrar la UI funcionando

SECCIÓN 4 — Dashboard del Vendedor `/marketplace/dashboard`

Archivo: `src/pages/VendorDashboardPage.tsx`

NO es el dashboard Bloomberg genérico — es el panel de control de ofertas del vendedor.

Sección 1 — Header con métricas rápidas (4 KPIs):

[Mis Ofertas Activas: 3] [Consultas esta semana: 7] [Deals en curso: 2] [Score reputación: 4.8★]

Sección 2 — Mis Ofertas Activas (tabla):

Columnas: Producto | NCM | Precio | Cantidad | Vistas | Consultas | Compatibilidad % | Estado | Acciones

Compatibilidad %: qué % del mercado puede operar esta oferta con su documentación actual (calculado en base a los perfiles que han visto la oferta)

Estado: badge Activa (verde) / Pausada (amber) / Cerrada (gris)

Acciones: Editar | Pausar | Ver consultas

Sección 3 — Pipeline de Deals (kanban horizontal):

4 columnas:

[Contacto iniciado] [Documentación] [Negociación] [Cerrado]
card deal card deal card deal

Cada deal card: nombre empresa, producto, valor USD, días en esta etapa, btn "Ver chat"

Sección 4 — Compatibilidad de mi perfil con el mercado:

Widget que muestra:

Con tu documentación actual (Score: 45/100) podés operar:

 62% de las ofertas activas del mercado

Desglosado por región: MERCOSUR 89% · Europa 34% · Asia 18%

Incentivo: "Completá [Certificado de Origen MERCOSUR] para acceder a 14 ofertas más"

Sección 5 — Score de Reputación:

- Rating general con estrellas
- Métricas: tiempo de respuesta promedio, deals cerrados, docs siempre completos, reseñas

REGLAS TÉCNICAS IMPORTANTES

1. **No hardcodear requisitos documentales** — siempre usar `POST /api/onboarding/requirements` que tiene el sistema de cache híbrido. Para renders de listas (el indicador en cada card del feed), usar solo el cache local de SQLite, nunca la IA en tiempo real.
2. **Loading states en todos los fetches** — el análisis documental puede tardar 1-2 segundos si Groq genera para una ruta nueva. Siempre mostrar spinner + texto contextual.
3. **El indicador de compatibilidad en las cards** del feed se calcula client-side comparando `userProfile.docsCompleted` (array de strings) contra los requisitos cacheados. Si la ruta no está en cache, mostrar "⌚ Verificando..." y fetchear en background.
4. **El chat NO necesita WebSockets para el MVP** — puede usar polling cada 3 segundos con `GET /api/chat/messages/:dealId`. El backend ya tiene la ruta básica.
5. **Vite proxy** ya configurado en `vite.config.ts` apuntando a `http://127.0.0.1:3001`. Todas las llamadas API deben ser relativas (`/api/...`).
6. **Design system:** usar exactamente las variables CSS del proyecto existente. No introducir nuevos colores ni fuentes. Los componentes del onboarding ya tienen los tokens definidos en `src/components/onboarding/ui.tsx`.
7. **El modal de costos** debe poder abrirse desde cualquier card del feed. Manejar el estado con `useState<Publication | null>(null)` en `MarketplacePage` y pasar al modal como prop.
8. **Filosofía UX — SIEMPRE comprensible:** cada número tiene una etiqueta, cada estado tiene un color, cada acción tiene una consecuencia visible. La plataforma enseña mientras opera — tooltips en términos técnicos (Incoterms, NCM, aranceles), explicaciones inline en lenguaje PyME.

ARCHIVOS A CREAR/MODIFICAR

CREAR:

`src/pages/MarketplacePage.tsx`

`src/pages/ChatPage.tsx`

`src/pages/VendorDashboardPage.tsx`

`src/components/marketplace/PublicationCard.tsx`

`src/components/marketplace/CostAnalysisModal.tsx`

`src/components/marketplace/NewPublicationBar.tsx`

src/components/marketplace/FilterPanel.tsx
src/components/marketplace/WorldTradePulse.tsx
src/components/chat/ChatWindow.tsx
src/components/chat/ParticipantsPanel.tsx
src/components/chat/DealTimeline.tsx

MODIFICAR:

src/App.tsx → agregar rutas /marketplace, /chat/:dealId, /marketplace/dashboard
backend/server.ts → agregar GET /api/chat/messages/:dealId, POST /api/chat/send

FLUJO COMPLETO DE NAVEGACIÓN

/marketplace

- clic "Ver Costos" → modal CostAnalysisModal (mismo componente)
- clic "Contactar" → /chat/:dealId (nuevo deal o existente)
- clic avatar propio → /marketplace/dashboard
- clic "Subir docs" → /onboarding (paso documentación) o modal inline

Che.Comex v2.0 — Jezabel Ayelén Villalba — San Lorenzo, Santa Fe, Argentina — Marzo 2026